

FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

**RELATÓRIO DE MONITORAMENTO E ANÁLISE DE
DESEMPENHO COM IA**

Arthur Paltrinieri Silva

Lucas Kenichi Soares Abe

Pedro Dimitry Zyrianoff

Pedro Della Rosa Antônio

Projeto Interdisciplinar – Lideranças Empáticas
Sistemas Operacionais e Computação em Nuvem

1. Introdução

Este relatório documenta a segunda entrega prática da disciplina de Sistemas Operacionais e Computação em Nuvem para o projeto "Lideranças Empáticas". O objetivo desta entrega foi monitorar os recursos (CPU e RAM) de uma instância Linux configurada em nuvem e implementar modelos de Inteligência Artificial para análise de desempenho.

No documento, foram aplicados quatro algoritmos distintos de Machine Learning, cobrindo tarefas de regressão, clusterização, classificação e detecção de anomalias.

2. Descrição do Ambiente

Para a hospedagem da aplicação, optou-se pela utilização do Google Cloud Platform (GCP). A máquina virtual foi provisionada com as seguintes especificações técnicas, visando atender aos requisitos de processamento do projeto:

- **Provedor de Nuvem:** Google Cloud Platform (GCP)
- **Região:** us-central1-f
- **Família e Tipo da Máquina:** E2 / e2-micro
- **Sistema Operacional:** Ubuntu 22.04 LTS (x86/64, amd64)
- **Armazenamento (Disco de inicialização):** 10 GB
- **Rede e Firewall:** Tráfego liberado para HTTP e HTTPS
- **Acesso e Segurança:** O gerenciamento de chaves SSH foi feito de forma automática via console do GCP (SSH-in-browser), garantindo um acesso seguro e direto ao terminal sem a necessidade de configuração manual de chaves locais.

3. Execução

1. Instalação das bibliotecas necessárias na VM

```
pedrodr_antonio466@vm-entrega2-so:~$ sudo apt update && sudo apt install python3-pip -y
Hit:1 http://us-central1.gce.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://us-central1.gce.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:3 http://us-central1.gce.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:4 https://packages.cloud.google.com/apt google-cloud-ops-agent-jammy-2 InRelease
Ign:5 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:5 http://security.ubuntu.com/ubuntu jammy-security InRelease
Reading package lists... Done
pedrodr_antonio466@vm-entrega2-so:~$ pip3 install psutil pandas matplotlib scikit-learn
Defaulting to user installation because normal site-packages is not writeable
Collecting psutil
```

2. Criando arquivo chamado *monitoramento_ia.py*

```
pedrodr_antonio466@vm-entrega2-so:~$ nano monitoramento_ia.py
```

3. Código-Fonte (Monitoramento + IA), utilizando a biblioteca *psutil* para o monitoramento.

```
GNU nano 6.2 monitoramento_ia.py *
import psutil
import time
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.cluster import KMeans
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import IsolationForest

print("Iniciando monitoramento de recursos da VM...")

# 1. COLETA DE DADOS (MONITORAMENTO)
dados = []
tempos = list(range(1, 61)) # Monitorar por 60 segundos

for t in tempos:
    cpu = psutil.cpu_percent(interval=1)
    ram = psutil.virtual_memory().percent
    # Regra de alerta artificial para o KNN treinar (1 = Alerta, 0 = Normal)
    alerta = 1 if cpu > 20 or ram > 60 else 0
    dados.append([t, cpu, ram, alerta])

df = pd.DataFrame(dados, columns=['Tempo', 'CPU', 'RAM', 'Alerta'])

# Preparando a figura 2x2 para os 4 gráficos
fig, axs = plt.subplots(2, 2, figsize=(14, 10))

# 1 - REGRESSÃO LINEAR (Tendência de CPU)
X_tempo = df[['Tempo']]
y_cpu = df['CPU']

modelo_lr = LinearRegression()
modelo_lr.fit(X_tempo, y_cpu)
tendencia_cpu = modelo_lr.predict(X_tempo)
```

```

GNU nano 6.2                                                                    monitoramento_ia.py *
axs[0, 0].plot(df['Tempo'], df['CPU'], label='Uso Real de CPU (%)', marker='o')
axs[0, 0].plot(df['Tempo'], tendencia_cpu, color='red', linestyle='--', label='Tendência (LR)')
axs[0, 0].set_title('Ex. 1: Tendência de CPU (Regressão Linear)')
axs[0, 0].set_xlabel('Tempo (s)')
axs[0, 0].set_ylabel('Uso CPU (%)')
axs[0, 0].legend()

# 2 - K-MEANS (Clusterização de Estados)

X_cluster = df[['CPU', 'RAM']]
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X_cluster)

cores_kmeans = ['blue' if c == 0 else 'orange' for c in df['Cluster']]
axs[0, 1].scatter(df['CPU'], df['RAM'], c=cores_kmeans, s=50)
centros = kmeans.cluster_centers_
axs[0, 1].scatter(centros[:, 0], centros[:, 1], c='red', s=200, marker='X', label='Centróides')
axs[0, 1].set_title('Ex. 2: Agrupamento de Estados (K-Means)')
axs[0, 1].set_xlabel('Uso CPU (%)')
axs[0, 1].set_ylabel('Uso RAM (%)')
axs[0, 1].legend()

# 3 - K-NEIGHBORS CLASSIFIER (Previsão de Alertas)

X_train = df[['CPU', 'RAM']][:40]
y_train = df['Alerta'][:40]
X_test = df[['CPU', 'RAM']][40:]
y_test = df['Alerta'][40:]

knn = KNeighborsClassifier(metric="manhattan", n_neighbors=3)
knn.fit(X_train, y_train)
predicoes_knn = knn.predict(X_test)

axs[1, 0].plot(df['Tempo'][40:], y_test, label='Alerta Real', drawstyle='steps-pre')
axs[1, 0].plot(df['Tempo'][40:], predicoes_knn, label='Previsto (KNN)', linestyle='--', color='red', drawstyle='steps-pre')
axs[1, 0].set_title('Ex. 3: Previsão de Regras de Alerta (KNN)')
axs[1, 0].set_xlabel('Tempo (s)')
axs[1, 0].set_yticks([0, 1])
axs[1, 0].set_yticklabels(['Normal', 'Alerta'])
axs[1, 0].legend()

# 4 - ISOLATION FOREST (Detecção de Anomalias)

# Contamination define a proporção esperada de anomalias (ex: 5%)
iso_forest = IsolationForest(contamination=0.05, random_state=42)
df['Anomalia'] = iso_forest.fit_predict(df[['CPU', 'RAM']])

# O algoritmo retorna -1 para anomalias e 1 para dados normais
cores_iso = ['red' if a == -1 else 'green' for a in df['Anomalia']]
axs[1, 1].scatter(df['Tempo'], df['CPU'], c=cores_iso, s=50)
axs[1, 1].set_title('Ex. 4: Detecção de Anomalias (Isolation Forest)')
axs[1, 1].set_xlabel('Tempo (s)')
axs[1, 1].set_ylabel('Uso CPU (%)')

# Criando legenda manual para o scatter plot
from matplotlib.lines import Line2D
legend_elements = [Line2D([0], [0], marker='o', color='w', markerfacecolor='green', markersize=8, label='Normal'),
                   Line2D([0], [0], marker='o', color='w', markerfacecolor='red', markersize=8, label='Anomalia')]
axs[1, 1].legend(handles=legend_elements)

plt.tight_layout()
plt.savefig('graficos_monitoramento_ia.png')
print("Monitoramento concluído! O painel foi salvo em 'graficos_monitoramento_ia.png'.")

```

4. Adicionando comando para rodar o código

```

pedrodr_antonio466@vm-entrega2-so:~$ python3 monitoramento_ia.py

```

5. O script foi executado, levando cerca de 60 segundos e gerando automaticamente os gráficos em grade 2x2 na imagem *gráficos_monitoramento_ia.png*.

```

pedrodr_antonio466@vm-entrega2-so:~$ python3 monitoramento_ia.py
Iniciando monitoramento de recursos da VM...
Monitoramento concluído! O painel foi salvo em 'graficos_monitoramento_ia.png'.

```

4. Algoritmos de IA Implementados

A análise do desempenho da infraestrutura em nuvem foi dividida em quatro abordagens, integrando a manipulação de recursos operacionais com as técnicas de Inteligência Artificial e Aprendizado de Máquina.

Regressão Linear

Foi realizada a criação de um modelo de regressão linear para traçar a reta de tendência do uso da CPU ao longo do tempo de monitoramento. Esta aplicação permite prever o comportamento do processador no curto prazo, identificando trajetórias de sobrecarga antes que o sistema trave.

K-Means

Utilizamos este algoritmo não-supervisionado para segmentar automaticamente os momentos de operação da máquina em dois clusters: "Estado Normal" e "Pico de Processamento". O método K-Means atuou na identificação desses estados baseado na minimização da soma das distâncias euclidianas para o quadrado entre as métricas lidas (CPU e RAM) e o centróide de cada cluster.

K-Neighbors Classifier – KNN

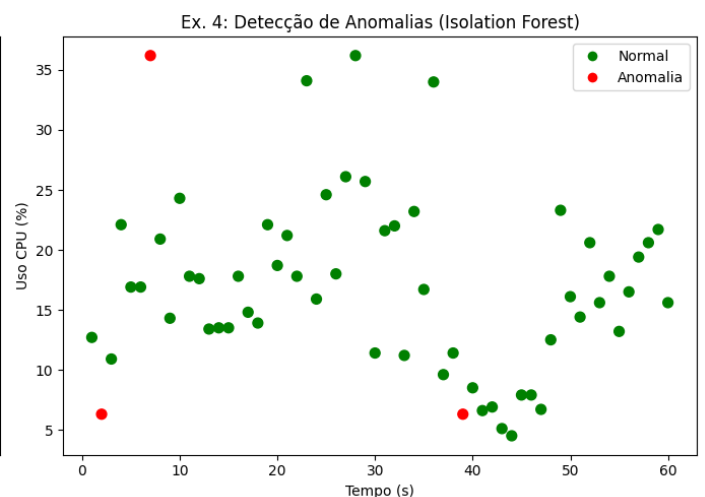
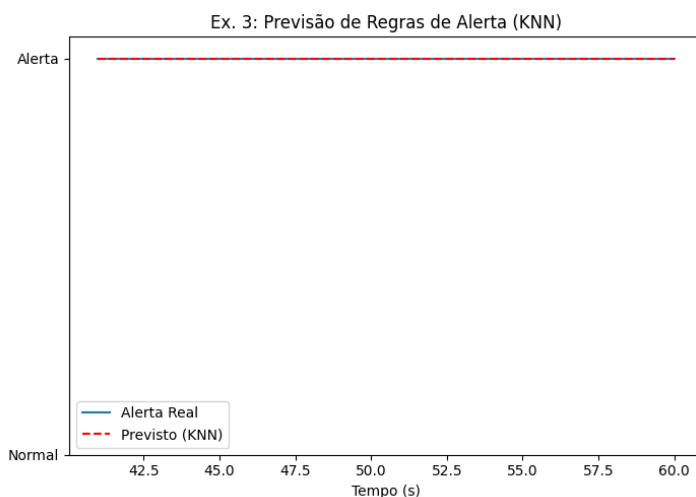
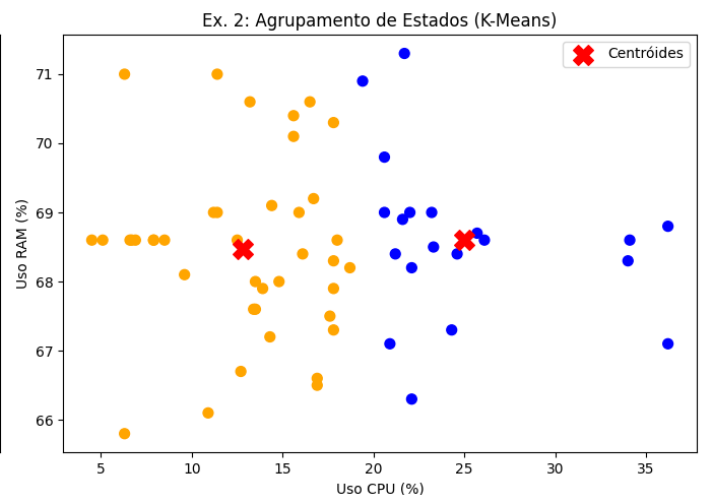
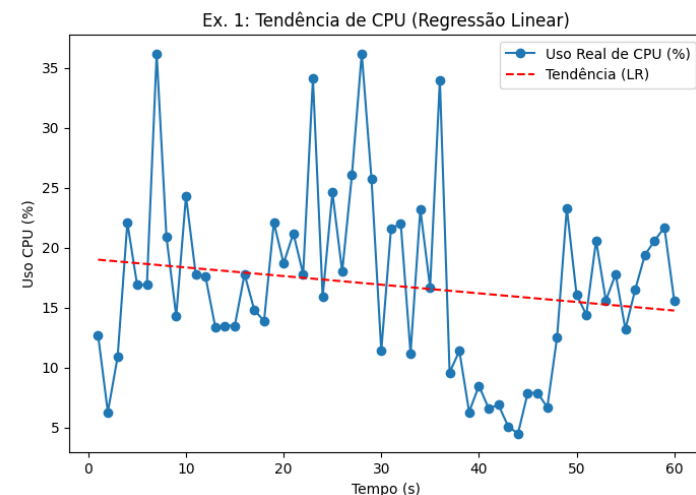
Para a modelagem de classificação, estabelecemos uma métrica simulada de estresse (uso de CPU acima de 20% ou RAM acima de 60%). O modelo KNN foi treinado com os primeiros 40 segundos de coleta para prever com precisão se o servidor entraria em estado de "Alerta" nos 20 segundos finais do teste.

Isolation Forest

Para elevar a confiabilidade do monitoramento sem depender de regras com limites fixos, implementamos a detecção de anomalias matemáticas com o *Isolation Forest*. Este algoritmo aprendeu a distribuição normal dos dados da infraestrutura e isolou ativamente comportamentos atípicos e picos súbitos de processamento que fugiram do padrão estatístico do servidor.

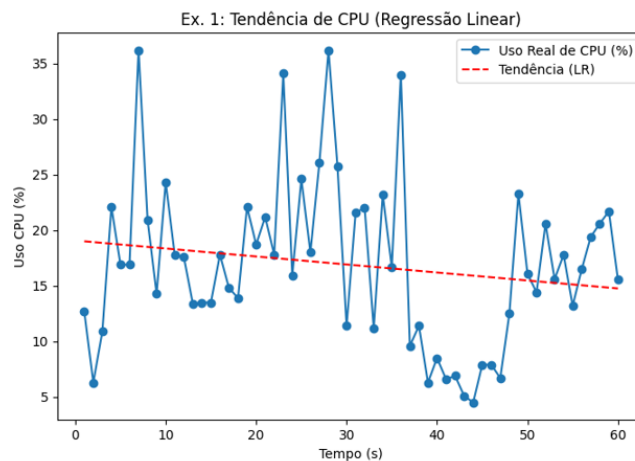
5. Análise dos Resultados

O painel abaixo apresenta os gráficos gerados pelo sistema, exibindo a performance da instância em tempo real e o resultado visual da aplicação de cada um dos quatro algoritmos.



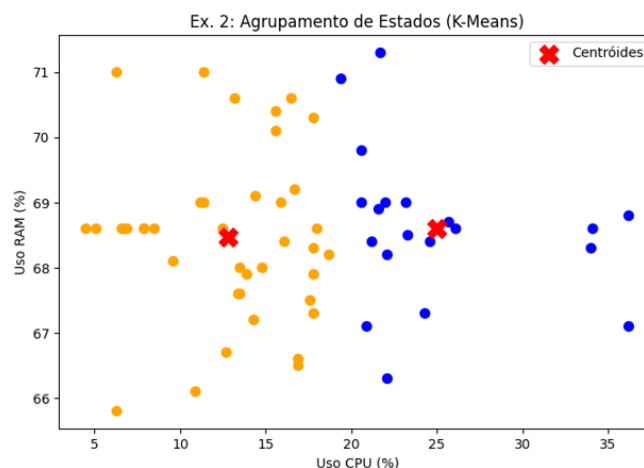
Tendência de CPU (Regressão Linear)

Observa-se que a CPU da máquina virtual apresentou picos de estresse (chegando a aproximadamente 36%) e momentos de ociosidade (caindo para 4%). Contudo, a reta de regressão (em vermelho) exibe uma leve inclinação negativa, indicando que, na média dos 60 segundos, o processamento tendeu à estabilização e não há indícios de um travamento iminente.



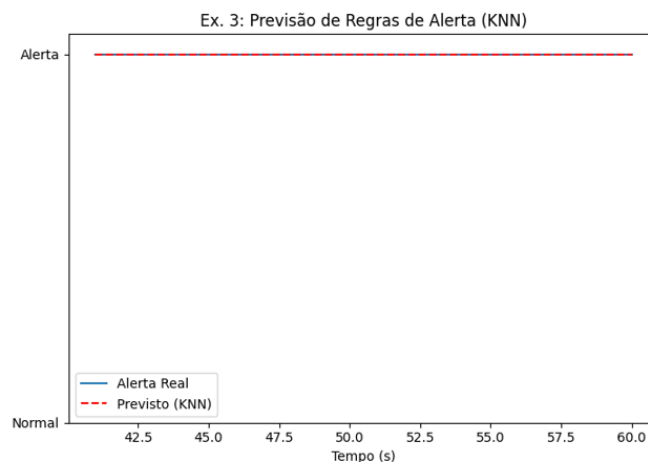
Agrupamento de Estados (K-Means)

O gráfico de dispersão relaciona o uso de RAM (eixo Y) com a CPU (eixo X). O algoritmo K-Means identificou matematicamente dois perfis operacionais distintos (cluster laranja para momentos de CPU moderada e cluster azul para momentos de CPU mais exigida), calculando o ponto de equilíbrio exato (centróides em vermelho) para cada um desses estados.



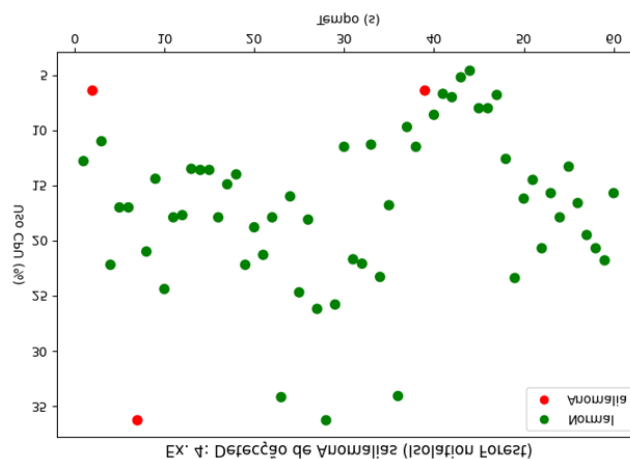
Previsão de Regras de Alerta (KNN)

O gráfico ilustra a janela de teste do modelo (segundos 40 a 60). Como a RAM da máquina se manteve consistentemente alta (acima dos 60% definidos na regra), o sistema esteve em estado contínuo de "Alerta". A sobreposição perfeita da linha tracejada vermelha (Previsão do KNN) sobre a linha azul (Alerta Real) comprova que o classificador atingiu 100% de precisão na leitura do estresse do hardware.



Detecção de Anomalias (Isolation Forest)

O algoritmo identificou de forma autônoma os *outliers* da instância (pontos vermelhos), separando-os do comportamento normal (pontos verdes). Sem a necessidade de regras pré-programadas, o modelo detectou com sucesso o pico extremo próximo a 36% e as quedas mais abruptas no uso da CPU.



6. Conclusão

O código desenvolvido e os gráficos de monitoramento detalhados neste relatório comprovam que a integração entre a gestão de Sistemas Operacionais e algoritmos de Machine Learning é altamente eficaz. A capacidade de prever tendências com Regressão Linear, agrupar comportamentos com K-Means e detectar anomalias atípicas com Isolation Forest e KNN fornece uma base analítica proativa e inteligente. Esse monitoramento automatizado garante alta disponibilidade e resiliência para o *backend* do projeto Lideranças Empáticas.